

Intelligenza Artificiale e Laboratorio

Planning, Sistemi a Regole e Incertezza

Roberto Micalizio

a.a. 2025/2026

Università degli Studi di Torino,
Dipartimento di Informatica



Graphplan

- **Grafo di pianificazione**

- struttura dati speciale utile per:
 - definire **euristiche domain-independent** (non dà garanzie sulla raggiungibilità di uno stato, ma fornisce una buona stima della distanza)
 - **generare** un piano (**graphplan**)
- il grafo astrae lo spazio di ricerca
- può essere costruito con un algoritmo polinomiale

Grafo di Pianificazione (GP):

- **Grafo orientato, aciclico, organizzato a livelli**

- livello iniziale S_0 contiene i letterali che valgono nello stato iniziale
(un nodo per ogni fluente)
- livello di azioni A_0 contiene le azioni ground che possono essere applicate ad S_0
(un nodo per ogni azione)
- in generale, S_i , A_i si alternano fino a che non si raggiunge una condizione di terminazione (nel seguito) e sono t.c:
 - S_i contiene tutti i letterali che **potrebbero** valere al tempo i
NB sia P sia $\neg P$ potrebbero appartenere ad S_i
 - A_i contiene tutte le azioni che **potrebbero** avere le precondizioni soddisfatte al tempo i

Grafo di Pianificazione (GP):

- **Grafo orientato, aciclico, organizzato a livelli**

- livello iniziale S_0 contiene i letterali che valgono nello stato iniziale
(un nodo per ogni fluente)
- livello di azioni A_0 contiene le azioni ground che possono essere applicate ad S_0
(un nodo per ogni azione)
- in generale, S_i , A_i si alternano fino a che non si raggiunge una condizione di terminazione (nel seguito) e sono t.c:
 - S_i contiene tutti i letterali che **potrebbero** valere al tempo i
NB sia P sia $\neg P$ potrebbero appartenere ad S_i
 - A_i contiene tutte le azioni che **potrebbero** avere le precondizioni soddisfatte al tempo i

NB GRAPHPLAN usa una rappresentazione logica del problema di planning, a differenza del Least-Commitment Planning che usa CSP

Grafo di Pianificazione (GP):

- **Grafo orientato, aciclico, organizzato a livelli**

- livello iniziale S_0 contiene i letterali che valgono nello stato iniziale
(un nodo per ogni fluente)
- livello di azioni A_0 contiene le azioni ground che possono essere applicate ad S_0
(un nodo per ogni azione)
- in generale, S_i , A_i si alternano fino a che non si raggiunge una condizione di terminazione (nel seguito) e sono t.c:
 - S_i contiene tutti i letterali che **potrebbero** valere al tempo i
NB sia P sia $\neg P$ potrebbero appartenere ad S_i
 - A_i contiene tutte le azioni che **potrebbero** avere le precondizioni soddisfatte al tempo i

NB GRAPHPLAN usa una rappresentazione logica del problema di planning, a differenza del Least-Commitment Planning che usa CSP

Grafo di Pianificazione (GP):

- **Grafo orientato, aciclico, organizzato a livelli**

- livello iniziale S_0 contiene i letterali che valgono nello stato iniziale
(un nodo per ogni fluente)
- livello di azioni A_0 contiene le azioni ground che possono essere applicate ad S_0
(un nodo per ogni azione)
- in generale, S_i , A_i si alternano fino a che non si raggiunge una condizione di terminazione (nel seguito) e sono t.c:
 - S_i contiene tutti i letterali che **potrebbero** valere al tempo i
NB sia P sia $\neg P$ potrebbero appartenere ad S_i
 - A_i contiene tutte le azioni che **potrebbero** avere le precondizioni soddisfatte al tempo i

NB GRAPHPLAN usa una rappresentazione logica del problema di planning, a differenza del Least-Commitment Planning che usa CSP

Grafo di Pianificazione (GP):

- **Considerazioni:**

- GP traccia solo un sottoinsieme delle possibili interazioni negative, quindi un letterale che appare in uno stato S_i per la prima volta, potrebbe di fatto essere producibile solo in uno stato successivo ad S_i . Ciononostante, i è una buona stima (ammissibile) per il letterale.
- GP è costruito a partire da azioni ground -> la **proposizionalizzazione** può generare un'esplosione di alternative. Ciononostante, GP è costruito in tempo polinomiale
- Ogni S_i è un **belief state**: codifica più stati possibili e alternativi

Considerazioni:

- Nella costruzione del grafo occorre tenere a mente che:
 - un letterale al tempo i può essere inteso *sia come preconditione di un'azione in A_i sia come atomo persistente* (i.e., non usato), in questo secondo caso la persistenza è realizzata da **un'azione speciale no-op** (sempre eseguibile)
 - in ogni livello S_i possono essere presenti link di **mutua esclusione** tra letterali (es. P e $\neg P$, o anche $Have(Cake)$ e $Eaten(Cake)$)
 - in ogni livello A_i possono essere presenti link di **mutua esclusione** tra azioni
 - GP cresce monotonicamente, quindi prima o poi si **livella**: due stati consecutivi S_i e S_{i+1} sono identici

Considerazioni:

- Nella costruzione del grafo occorre tenere a mente che:
 - un letterale al tempo i può essere inteso *sia come preconditione di un'azione in A_i sia come atomo persistente* (i.e., non usato), in questo secondo caso la persistenza è realizzata da **un'azione speciale no-op** (sempre eseguibile)
 - in ogni livello S_i possono essere presenti link di **mutua esclusione** tra letterali (es. P e $\neg P$, o anche $Have(Cake)$ e $Eaten(Cake)$)
 - in ogni livello A_i possono essere presenti link di **mutua esclusione** tra azioni
 - GP cresce monotonicamente, quindi prima o poi si **livella**: due stati consecutivi S_i e S_{i+1} sono identici

Considerazioni:

- Nella costruzione del grafo occorre tenere a mente che:
 - un letterale al tempo i può essere inteso *sia come preconditione di un'azione* in A_i *sia come atomo persistente* (i.e., non usato), in questo secondo caso la persistenza è realizzata da **un'azione speciale no-op** (sempre eseguibile)
 - in ogni livello S_i possono essere presenti link di **mutua esclusione** tra letterali (es. P e $\neg P$, o anche $Have(Cake)$ e $Eaten(Cake)$)
 - in ogni livello A_i possono essere presenti link di **mutua esclusione** tra azioni
 - GP cresce monotonicamente, quindi prima o poi si **livella**: due stati consecutivi S_i e S_{i+1} sono identici

Considerazioni:

- Nella costruzione del grafo occorre tenere a mente che:
 - un letterale al tempo i può essere inteso *sia come preconditione di un'azione* in A_i *sia come atomo persistente* (i.e., non usato), in questo secondo caso la persistenza è realizzata da **un'azione speciale no-op** (sempre eseguibile)
 - in ogni livello S_i possono essere presenti link di **mutua esclusione** tra letterali (es. P e $\neg P$, o anche $Have(Cake)$ e $Eaten(Cake)$)
 - in ogni livello A_i possono essere presenti link di **mutua esclusione** tra azioni
 - GP cresce monotonicamente, quindi prima o poi si **livella**: due stati consecutivi S_i e S_{i+1} sono identici

Init(*Have*(*Cake*))
Goal(*Have*(*Cake*) $\wedge$ *Eaten*(*Cake*))
Action(*Eat*(*Cake*))
 PRECOND: *Have*(*Cake*)
 EFFECT: $\neg$ *Have*(*Cake*) $\wedge$ *Eaten*(*Cake*)
Action(*Bake*(*Cake*))
 PRECOND: $\neg$ *Have*(*Cake*)
 EFFECT: *Have*(*Cake*)

Figure 10.7 The “have cake and eat cake too” problem.

Grafi di pianificazione - Esempio

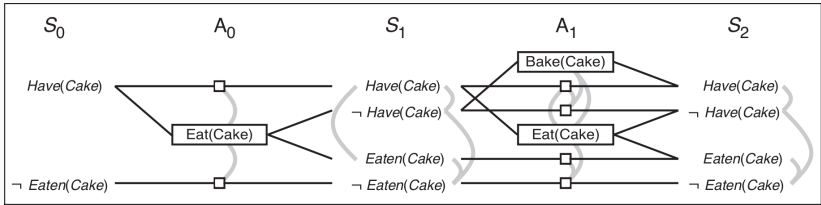


Figure 10.8 The planning graph for the “have cake and eat cake too” problem up to level S_2 . Rectangles indicate actions (small squares indicate persistence actions), and straight lines indicate preconditions and effects. Mutex links are shown as curved gray lines. Not all mutex links are shown, because the graph would be too cluttered. In general, if two literals are mutex at S_i , then the persistence actions for those literals will be mutex at A_i and we need not draw that mutex link.

- **Mutua esclusione tra due azioni in un dato livello A_i**
 - **Effetti inconsistenti**: un'azione nega gli effetti dell'altra
 - Es. in A_0 l'azione $Eat(Cake)$ è inconsistente con l'azione di persistenza del letterale $Have(Cake)$ perché gli effetti dell'una sono negati dall'altra
 - **Interferenza**: uno degli effetti di un'azione è la negazione di una precondizione dell'altra
 - Es. in A_0 l'azione $Eat(Cake)$ interferisce con l'azione di persistenza del letterale $Have(Cake)$ perché l'effetto di $Eat(Cake)$ nega le precondizioni della persistenza di $Have(Cake)$
 - **Competizione delle precondizioni**: una delle precondizioni di un'azione è mutuamente esclusiva con le precondizioni dell'altra.
 - Es. $Bake(Cake)$ e $Eat(Cake)$ sono mutex perché competono per il valore della precondizione $Have(Cake)$

- Mutua esclusione tra due letterali in un dato livello S_i
 - *Complementarity*: se uno è la negazione dell'altro

- **Mutua esclusione tra due letterali in un dato livello S_i**
 - *Complementarity*: se uno è la negazione dell'altro
 - *Inconsistent Support*: se ogni possibile coppia di azioni al livello A_{i-1} che producono i due letterali sono mutuamente esclusivi.

- **Mutua esclusione tra due letterali in un dato livello S_i**
 - *Complementarity*: se uno è la negazione dell'altro
 - *Inconsistent Support*: se ogni possibile coppia di azioni al livello A_{i-1} che producono i due letterali sono mutuamente esclusivi.
 - Es. *Have(Cake)* e *Eaten(Cake)* sono mutex in S_1 : l'unico modo per produrre *Have(Cake)* è in mutex con l'unico modo per produrre *Eaten(Cake)*
In S_2 i due letterali non sono più in mutex perché l'azione *Bake(Cake)* (che produce *Have(Cake)*) non è in mutex con l'azione di persistenza per *Eaten(Cake)*

Costruzione di un GP è polinomiale nella dimensione del problema

- l : # letterali
- a : # azioni
- ogni livello S_i ha non più di l nodi e l^2 link di mutex tra letterali
- ogni livello A_i non ha più di $l + a$ nodi (inclusi i no-op) e $(a + l)^2$ link di mutex
- un grafo con n livelli ha dimensione $O(n(a + l)^2)$ e la stessa complessità per costruirlo

Osservazione

- Se un letterale del goal non compare in nessun livello allora il goal non è raggiungibile
- **MA** se anche tutti i letterali del goal compaiono in un qualche livello del GP non vuol dire che esiste una soluzione, **FORSE il problema è risolvibile**

Osservazione

- Se un letterale del goal non compare in nessun livello allora il goal non è raggiungibile
- **MA** se anche tutti i letterali del goal compaiono in un qualche livello del GP non vuol dire che esiste una soluzione, **FORSE il problema è risolvibile**

Euristiche per stimare il costo di un singolo letterale

- profondità del livello in cui il letterale compare per la prima volta
 - la stima è certamente ammissibile ma non molto precisa
- lunghezza del piano serializzato estratto dal GP

Euristiche per stimare il costo di una congiunzione di letterali

- **max-level:** prendere il massimo livello tra quelli in cui un letterale del goal compare per la prima volta (equivale a dire, il primo livello in cui compaiono *tutti i letterali del goal*, anche con possibili vincoli di mutua esclusione)
- **somma dei livelli:** assume indipendenza dei letterali del goal, non è ammissibile
- **livello di insieme:** profondità del livello in cui tutti i letterali del goal compaiono senza che alcuna coppia di essi sia in mutua esclusione
 - è ammissibile, funziona bene quando i letterali del goal non sono indipendenti tra loro
 - ignora le dipendenze tra tre o più letterali

Teorema

Se esiste un piano valido allora questo è un sottografo del grafo di pianificazione.

Teorema

Se esiste un piano valido allora questo è un sottografo del grafo di pianificazione.

Come estrarre una soluzione da un GP in modo efficiente?

I vincoli di mutua esclusione tagliano alcune strade sicuramente sbagliate, ma rimangono aperti ancora diversi punti di scelta.

Intuitivamente:

1. Espandi il grafo un livello per volta fino a quando tutti gli atomi del goal non compaiono all'interno dell' i -esimo stato
senza che vi siano vincoli di mutua esclusione tra loro
2. Invoca EXTRACT-SOLUTION per cercare un piano all'interno del grafo
 - se EXTRACT-SOLUTION trova una soluzione termina con successo
 - altrimenti, se non c'è ragione per proseguire perché la soluzione non esiste certamente, termina con insuccesso
 - altrimenti, vai al passo 3
3. espandi il grafo ancora di un livello e torna al passo 2

```
function GRAPHPLAN(problem) returns solution or failure  
graph  $\leftarrow$  INITIAL-PLANNING-GRAPH(problem)  
goals  $\leftarrow$  CONJUNCTS(problem.GOAL)  
nogoods  $\leftarrow$  an empty hash table  
for tl = 0 to  $\infty$  do  
    if goals all non-mutex in  $S_t$  of graph then  
        solution  $\leftarrow$  EXTRACT-SOLUTION(graph, goals, NUMLEVELS(graph), nogoods)  
        if solution  $\neq$  failure then return solution  
    if graph and nogoods have both leveled off then return failure  
    graph  $\leftarrow$  EXPAND-GRAPH(graph, problem)
```

Figure 10.9 The GRAPHPLAN algorithm. GRAPHPLAN calls EXPAND-GRAPH to add a level until either a solution is found by EXTRACT-SOLUTION, or no solution is possible.

- L'estrazione di un piano da un grafo di pianificazione (GP) corrisponde ad una ricerca (backward) in un sottografo AND/OR del GP:
 - **rami OR**: sono gli archi che dalle azioni ad un livello A_{i-1} producono un letterale p in un goal g al livello S_i
 - **rami AND**: sono gli archi che dagli atomi ad un livello S_i rappresentano le precondizioni per un'azione al livello A_i

- **Extract-Solution** sfrutta una funzione di supporto **GP-Search**
- **Extract-Solution** e **GP-Search** sono mutuamente ricorsive
- Intuizione:
 - **Extract-Solution** invoca **GP-Search** su un livello di azioni e su un sottogoal (regredito dal goal del problema)
 - **GP-Search** pianifica per il solo livello su cui è invocato (cerca azioni rilevanti per il sottogoal che non siano tra loro in mutua esclusione) e se ha successo, invoca **Extract-Solution** sul livello di stato precedente

```
EXTRACT-SOLUTION( $G, g, i$ )  
  if  $i = 0$  then return  $\langle \rangle$   
  if  $g \in \text{no-good}(i)$  then return failure  
   $\pi \leftarrow \text{GP-Search}(G, g, \emptyset, i - 1)$   
  if  $\pi \neq \text{failure}$  then return  $\pi$   
   $\text{no-good}(i) \leftarrow \text{no-good}(i) \cup \{g\}$   
  return failure
```

- G il grafo di pianificazione
- g il sottogoal su cui la funzione è invocata
- i la profondità di un livello di stato S_i
- Inizialmente:
 - g è il goal del problema di pianificazione
 - i è la profondità massima a cui G è stato espanso

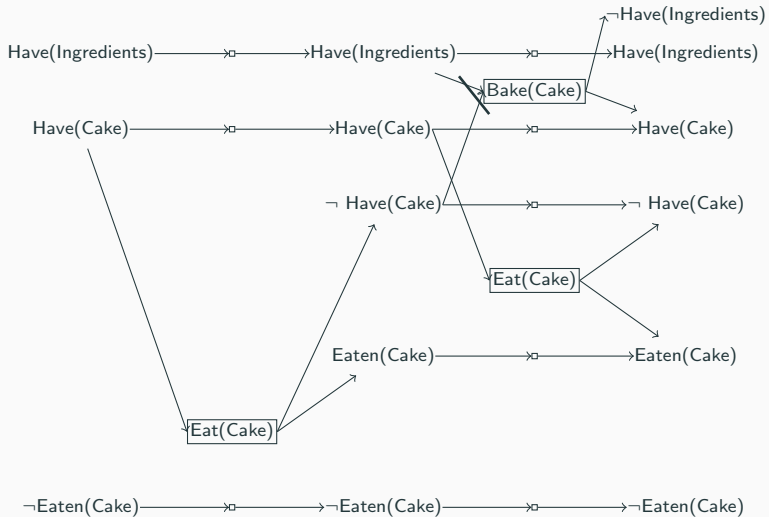
```
GP-SEARCH( $G, g, \pi_i, i$ )  
  if  $g = \emptyset$  then do  
     $\Pi \leftarrow \text{EXTRACT-SOLUTION}(G, \bigcup \{precond(a) \mid \forall a \in \pi_i\}, i)$   
    if  $\Pi = \text{failure}$  then return failure  
    return  $\Pi \cup \pi_i$   
  else  
    select any  $p \in g$   
     $resolvers \leftarrow \{a \in A_i \mid p \in effects^+(a) \text{ and } \forall b \in \pi_i : (a, b) \notin \mu A_i\}$   
    if  $resolvers = \emptyset$  return failure  
    nondeterministically choose a resolver  $a$  for  $p$   
    return GP-SEARCH( $G, g - effects^+(a), \pi_i \cup \{a\}, i$ )
```

- G grafo di pianificazione
- g insieme di sottogoal ancora da risolvere al livello i
- π_i piano in costruzione al livello i

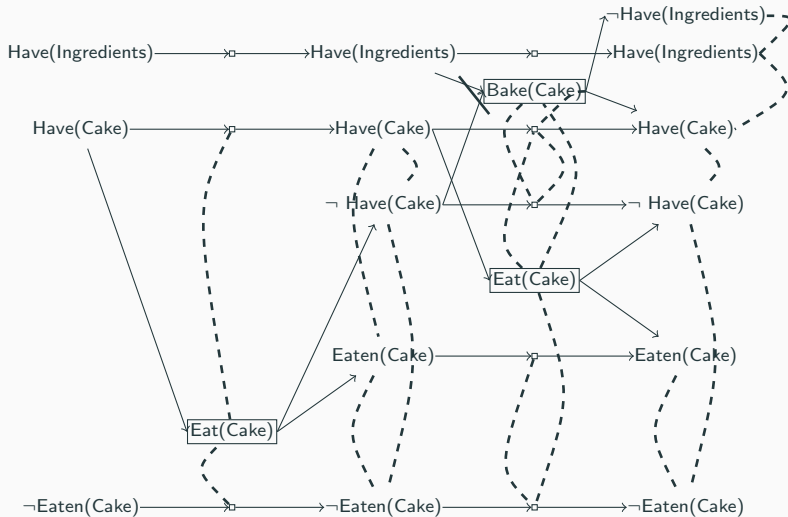
```
GP-SEARCH( $G, g, \pi_i, i$ )  
  if  $g = \emptyset$  then do  
     $\Pi \leftarrow \text{EXTRACT-SOLUTION}(G, \bigcup \{precond(a) | \forall a \in \pi_i\}, i)$   
    if  $\Pi = \text{failure}$  then return failure  
    return  $\Pi \cup \pi_i$   
  else  
    select any  $p \in g$   
     $resolvers \leftarrow \{a \in A_i | p \in effects^+(a) \text{ and } \forall b \in \pi_i : (a, b) \notin \mu A_i\}$   
    if  $resolvers = \emptyset$  return failure  
    nondeterministically choose a resolver  $a$  for  $p$   
    return GP-SEARCH( $G, g - effects^+(a), \pi_i \cup \{a\}, i$ )
```

- μA_i contiene le coppie di azioni al livello i tra le quali sussiste un vincolo di mutua esclusione

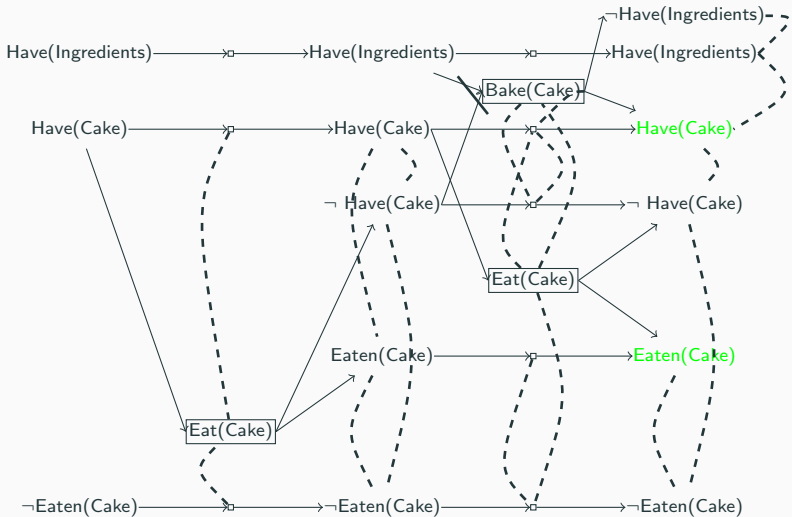
Esempio



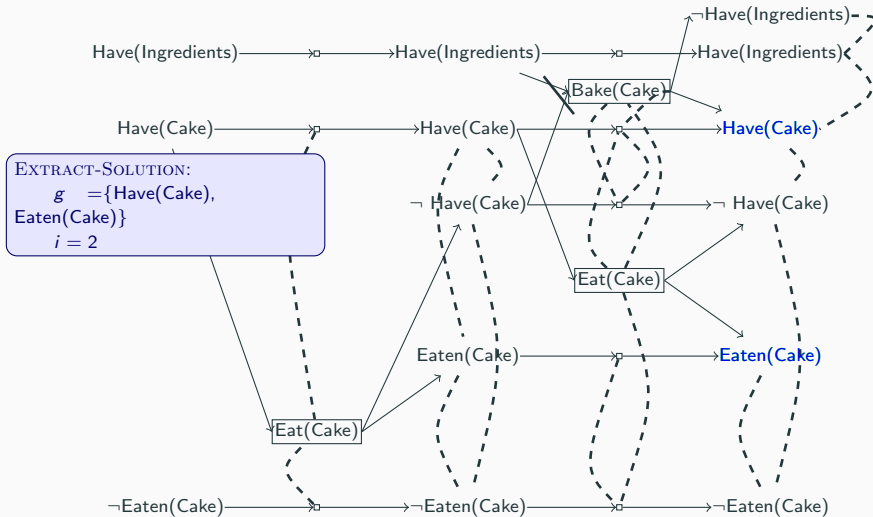
Esempio



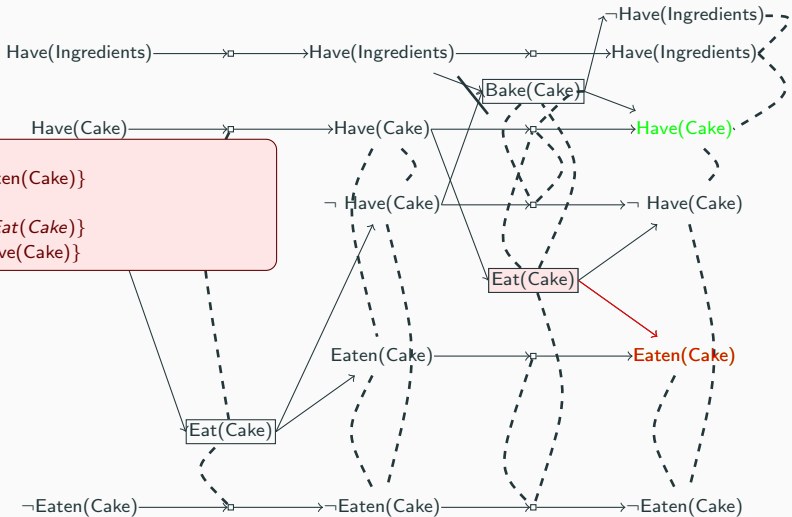
Esempio



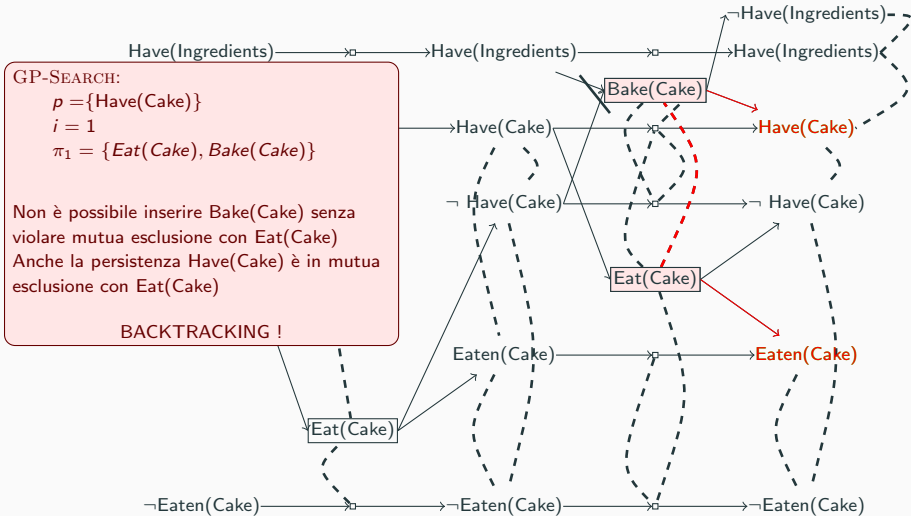
Esempio



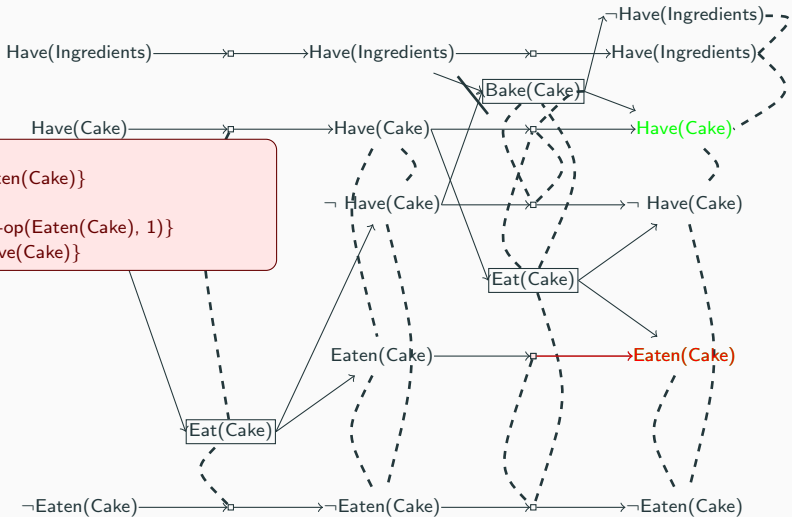
Esempio



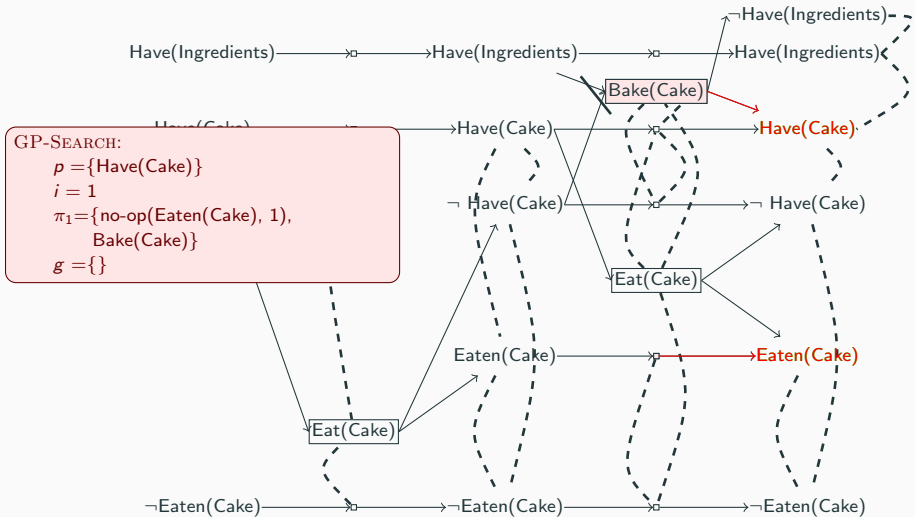
Esempio



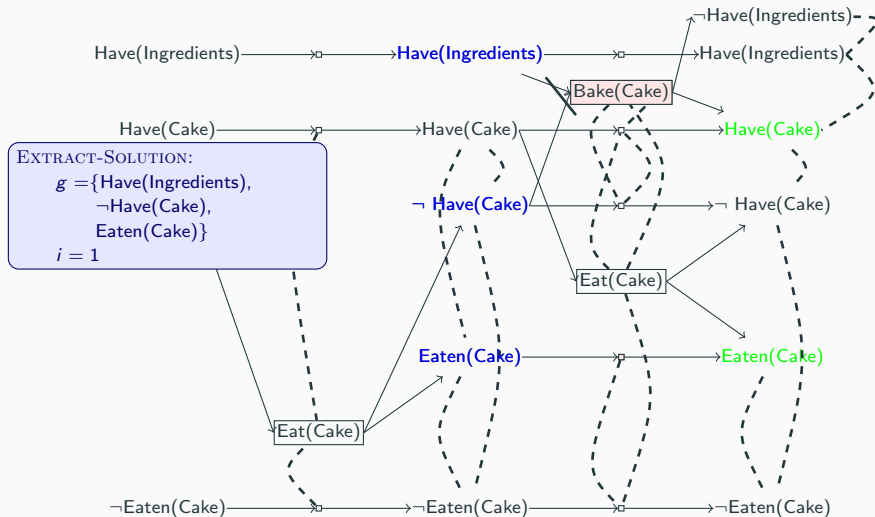
Esempio



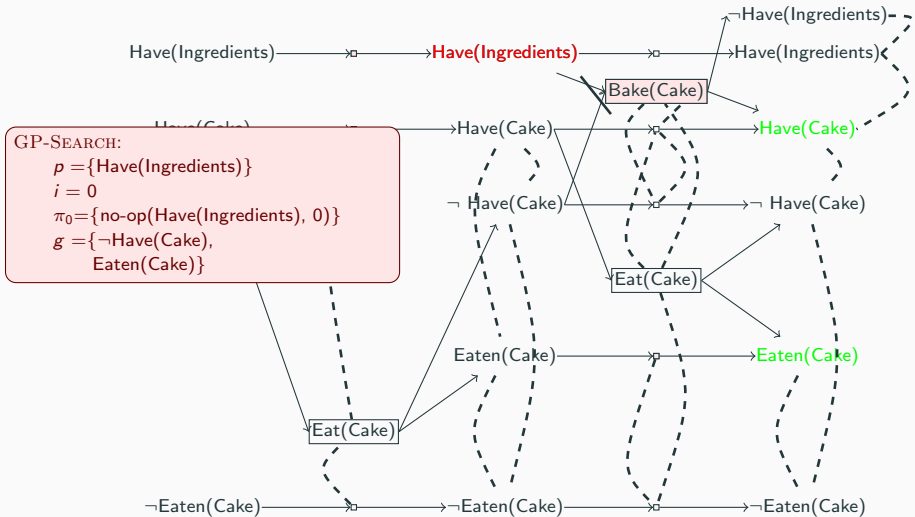
Esempio



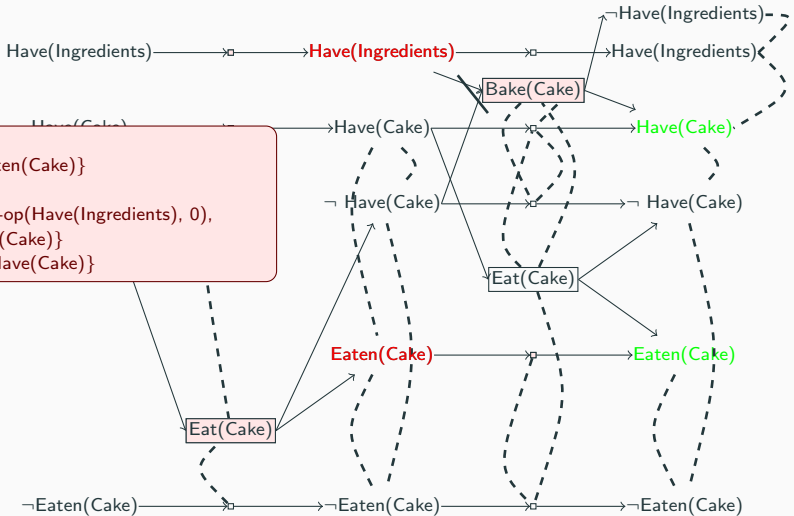
Esempio



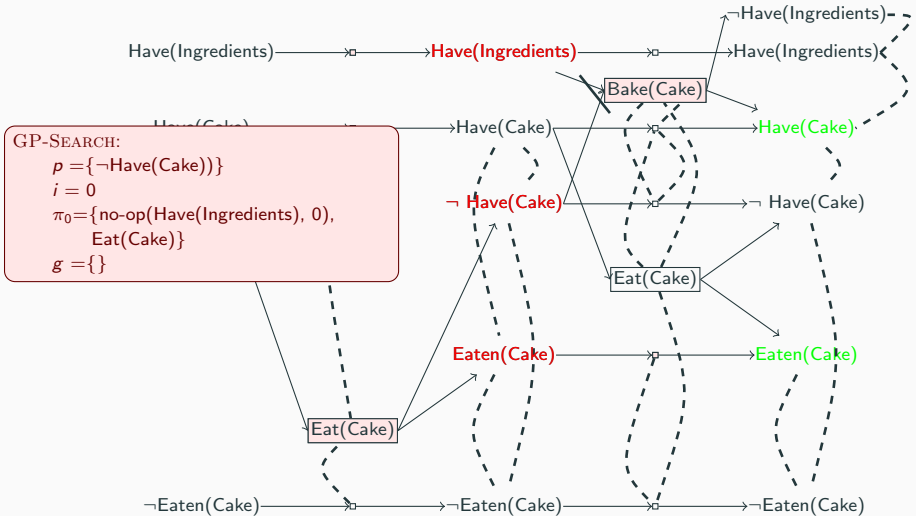
Esempio



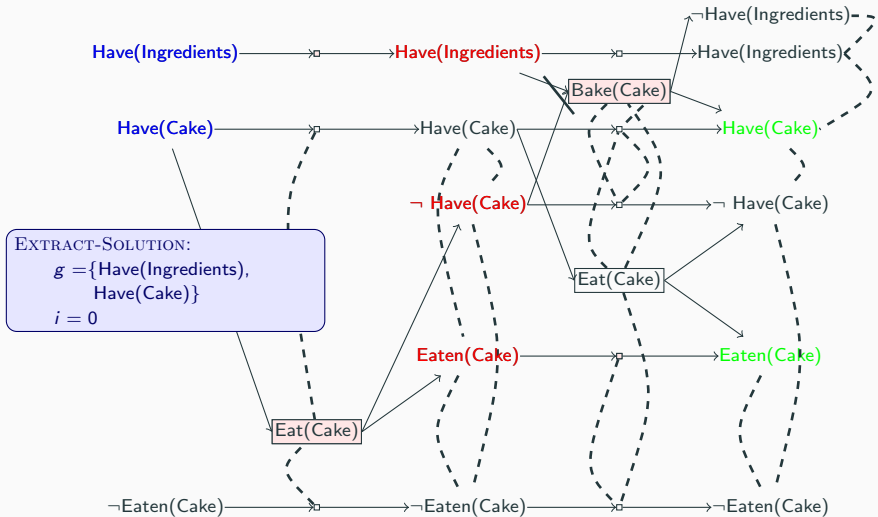
Esempio



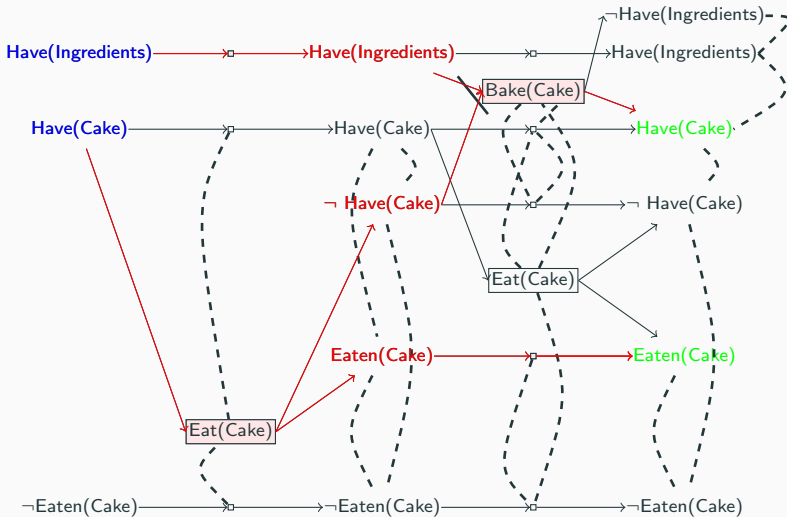
Esempio



Esempio



Esempio



Extract-Solution

- è un algoritmo **backward** di ricerca all'interno del grafo di pianificazione
- in generale questa ricerca potrebbe essere intrattabile e si ricorre quindi all'uso di euristiche
- un modo semplice per implementare questa funzione è un algoritmo greedy:
 - dato un insieme i letterali che compaiono nel goal
 - scegliere il letterale con il costo di livello più alto
 - soddisfare il letterale scegliendo l'azione con le precondizioni più "facili" (es. quella in cui la somma (o il massimo) dei costi di livello delle sue precondizioni è minima)
- ogni volta che EXTRACT-SOLUTION applicata a certi letterali L e ad un livello lev fallisce, la coppia (L, lev) viene aggiunta nell'insieme *no-good* -> ricerche successive applicate alla stessa coppia falliranno immediatamente.

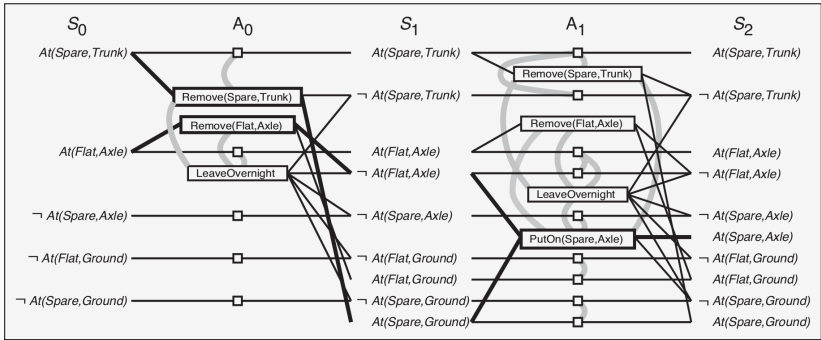


Figure 10.10 The planning graph for the spare tire problem after expansion to level S_2 . Mutex links are shown as gray lines. Not all links are shown, because the graph would be too cluttered if we showed them all. The solution is indicated by bold lines and outlines.

- **Osservazione** L'espansione del GP non si interrompe anche quando il grafo si è livellato. **Perché?**
- Un livello di stato può essere il risultato dell'applicazione di più azioni in parallelo
- Quando l'esecuzione è però sequenziale, l'applicazione multipla di azioni riduce la dimensione del grafo, che deve quindi essere esteso.

graphplan - Terminazione

- **Osservazione** L'espansione del GP non si interrompe anche quando il grafo si è livellato. **Perché?**
- Un livello di stato può essere il risultato dell'applicazione di più azioni in parallelo
- Quando l'esecuzione è però sequenziale, l'applicazione multipla di azioni riduce la dimensione del grafo, che deve quindi essere esteso.

Esempio

- Un trasporto aereo deve spostare n pacchi da A a B
- L'aereo può trasportare un solo pacco per volta
- Il GP corrispondente si livella a profondità 4, infatti:
 - livello 1 di azioni: una load per ogni pacco
 - livello 2 di azioni: un'unica fly da A a B
 - livello 3 di azioni: una unload per ogni pacco
- il livello di stato ottenuto è quello in cui tutti i pacchi sono in B
- **MA** da questo GP non è possibile estrarre una soluzione feasible
- occorre quindi continuare ad espandere il grafo per $4n - 1$ volte: load, fly, unload e fly-back (tranne che per l'ultimo pacco)

- Fino a quando continuare ad espandere il grafo dopo che si è livellato?
 - Se EXTRACT-SOLUTION non trova una soluzione, deve esistere un sottoinsieme di atomi del goal che non sono raggiungibili -> sono quindi marcati come *no-good*
 - Allora, se è possibile che vi siano **meno** *no-good* al livello successivo dobbiamo continuare ad espandere
 - Altrimenti, quando sia il grafo sia i *no-good* si sono livellati senza che una soluzione sia stata trovata, si può concludere con fallimento

- Affinché il criterio di terminazione enunciato sia utilizzabile dobbiamo dimostrare che:
 - i *no-good* si livellano sempre (cioè decrescono ad ogni estensione del grafo fino ad una certa soglia oltre la quale non decrescono più)
 - quando i *no-good* e il grafo sono livellati, allora si può concludere che la soluzione non esiste

- La dimostrazione si basa su proprietà **monotone** dei grafi di pianificazione:
 - i **letterali crescono monotonicamente**: una volta che un letterale compare in un livello sarà presente in tutti i livelli successivi (no-op)
 - le **azioni crescono monotonicamente**: una volta che un'azione compare in un livello sarà presente anche nei livelli successivi (conseguenza della proprietà precedente)
 - le **relazioni di mutex decrescono monotonicamente**: se due azioni (o due letterali) sono in mutex in un livello, allora lo sono anche in tutti i livelli precedenti, **ma** potrebbero non essere in mutex in un qualche livello successivo
 - i **no-goods decrescono monotonicamente**: se un sottoinsieme di atomi del goal non è raggiungibile in un dato livello di stato S_i , allora non lo è nemmeno in nessuno dei livelli precedenti, però potrà diventarlo in qualche livello futuro (conseguenza delle due proprietà precedenti)

- La dimostrazione si basa su proprietà **monotone** dei grafi di pianificazione:
 - i **letterali crescono monotonicamente**: una volta che un letterale compare in un livello sarà presente in tutti i livelli successivi (no-op)
 - le **azioni crescono monotonicamente**: una volta che un'azione compare in un livello sarà presente anche nei livelli successivi (conseguenza della proprietà precedente)
 - le **relazioni di mutex decrescono monotonicamente**: se due azioni (o due letterali) sono in mutex in un livello, allora lo sono anche in tutti i livelli precedenti, **ma** potrebbero non essere in mutex in un qualche livello successivo
 - i **no-goods decrescono monotonicamente**: se un sottoinsieme di atomi del goal non è raggiungibile in un dato livello di stato S_i , allora non lo è nemmeno in nessuno dei livelli precedenti, però potrà diventarlo in qualche livello futuro (conseguenza delle due proprietà precedenti)

- La dimostrazione si basa su proprietà **monotone** dei grafi di pianificazione:
 - i **letterali crescono monotonicamente**: una volta che un letterale compare in un livello sarà presente in tutti i livelli successivi (no-op)
 - le **azioni crescono monotonicamente**: una volta che un'azione compare in un livello sarà presente anche nei livelli successivi (conseguenza della proprietà precedente)
 - le **relazioni di mutex decrescono monotonicamente**: se due azioni (o due letterali) sono in mutex in un livello, allora lo sono anche in tutti i livelli precedenti, **ma** potrebbero non essere in mutex in un qualche livello successivo
 - i **no-goods decrescono monotonicamente**: se un sottoinsieme di atomi del goal non è raggiungibile in un dato livello di stato S_i , allora non lo è nemmeno in nessuno dei livelli precedenti, però potrà diventarlo in qualche livello futuro (conseguenza delle due proprietà precedenti)

- La dimostrazione si basa su proprietà **monotone** dei grafi di pianificazione:
 - i **letterali crescono monotonicamente**: una volta che un letterale compare in un livello sarà presente in tutti i livelli successivi (no-op)
 - le **azioni crescono monotonicamente**: una volta che un'azione compare in un livello sarà presente anche nei livelli successivi (conseguenza della proprietà precedente)
 - le **relazioni di mutex decrescono monotonicamente**: se due azioni (o due letterali) sono in mutex in un livello, allora lo sono anche in tutti i livelli precedenti, **ma** potrebbero non essere in mutex in un qualche livello successivo
 - i **no-goods decrescono monotonicamente**: se un sottoinsieme di atomi del goal non è raggiungibile in un dato livello di stato S_i , allora non lo è nemmeno in nessuno dei livelli precedenti, però potrà diventarlo in qualche livello futuro (conseguenza delle due proprietà precedenti)

- dimostrazione che il grafo e i no-goods si livellano e criterio di terminazione:
 1. dato che le azioni e i letterali crescono monotonicamente, e dato che sono entrambi insiemi finiti, *deve esistere un livello in cui l'insieme dei letterali è uguale a quello dei letterali del livello precedente*
 2. poiché i mutex e no-good decrescono monotonicamente e poiché non possono esserci meno di zero mutex e no-good, *deve esistere un livello con lo stesso numero di mutex e no-good del precedente*
 3. una volta che il grafo ha raggiunto la situazione in cui sia i livelli che i no-good si sono livellati, *se uno dei letterali del goal è mancante o in mutex con un altro letterale del goal, allora si può concludere che non esiste una soluzione*

- dimostrazione che il grafo e i no-goods si livellano e criterio di terminazione:
 1. dato che le azioni e i letterali crescono monotonicamente, e dato che sono entrambi insiemi finiti, *deve esistere un livello in cui l'insieme dei letterali è uguale a quello dei letterali del livello precedente*
 2. poiché i mutex e no-good decrescono monotonicamente e poiché non possono esserci meno di zero mutex e no-good, *deve esistere un livello con lo stesso numero di mutex e no-good del precedente*
 3. una volta che il grafo ha raggiunto la situazione in cui sia i livelli che i no-good si sono livellati, *se uno dei letterali del goal è mancante o in mutex con un altro letterale del goal, allora si può concludere che non esiste una soluzione*

- dimostrazione che il grafo e i no-goods si livellano e criterio di terminazione:
 1. dato che le azioni e i letterali crescono monotonicamente, e dato che sono entrambi insiemi finiti, *deve esistere un livello in cui l'insieme dei letterali è uguale a quello dei letterali del livello precedente*
 2. poiché i mutex e no-good decrescono monotonicamente e poiché non possono esserci meno di zero mutex e no-good, *deve esistere un livello con lo stesso numero di mutex e no-good del precedente*
 3. una volta che il grafo ha raggiunto la situazione in cui sia i livelli che i no-good si sono livellati, *se uno dei letterali del goal è mancante o in mutex con un altro letterale del goal, allora si può concludere che non esiste una soluzione*

graphplan è corretto e completo

Restituisce *failure* solo quando il problema di pianificazione non è risolubile, altrimenti restituisce una soluzione espressa come una **sequenza di insiemi di azioni**. L'algoritmo termina sempre anche quando la soluzione non esiste.

graphplan è un Partial-Order Planner (POP)

- azioni allo stesso livello (non legate da vincoli di mutua esclusione per costruzione) possono essere eseguite in parallelo
- qualunque linearizzazione delle azioni che rispetti i vincoli di ordinamento imposti dai livelli è una possibile soluzione “classica”

NB anche PSP è un POP!

Euristiche per la Pianificazione

Euristiche nel Planning

- La ricerca euristica è l'approccio più comune per la risoluzione di problemi di pianificazione
- Infatti, molti pianificatori consistono di :

algoritmo di ricerca + funzione euristica

- l'algoritmo include scelte non deterministiche e meccanismi di backtracking
- l'euristica cerca di focalizzare la ricerca sul goal guidando le scelte non deterministiche
- una funzione euristica stima la “bontà” di una scelta:
 - **progression**: la scelta è il “next state” s stima come distanza da s al goal G
 - **regression**: la scelta è il “next sub-goal” sg : stima come distanza da sg allo stato iniziale I

Il Principio del “Relaxing”

Approccio generale per costruire una funzione euristica

- Definire un problema analogo all'originale ma rilassato (i.e., relaxed), cioè semplificato
- Risolvere il problema rilassato (in maniera ottimale, se possibile)
- Usare il costo della soluzione per il problema rilassato come euristica nel problema originale

Osservazioni

- La funzione euristica viene invocata per ogni stato dello spazio di ricerca considerato dall'algoritmo, deve quindi essere efficiente
- L'euristica sarà tanto più affidabile quanto il problema rilassato sarà vicino al problema reale, d'altra parte tenderà ad essere più costosa

Possibili tecniche di rilassamento di un problema

- **Delete relaxation**: cancellazione degli effetti negativi dalle azioni
- **Astrazione**: considerare un problema più piccolo (es. con meno oggetti), ignorare alcune differenze tra stati così che collassino in un unico stato (spazio di ricerca ridotto)
- **Landmark**: è un insieme di azioni (o fatti) t.c. ogni soluzione deve avere almeno un'azione di quest'insieme, sono usati per definire euristiche che fanno un safe pruning dello spazio di ricerca

Utilizzo del problema rilassato

- Risolvi in modo **ottimo** il problema rilassato, il costo corrisponde all'euristica, problema computazionale
- Risolvere in modo **subottimo** il problema rilassato
- **Stimare** il costo della soluzione al problema rilassato senza risolverlo

EURISTICHE PER ALGORITMI DI RICERCA NELLO SPAZIO DEGLI STATI

Ignorare effetti negativi: Ricerca in Avanti

- $\gamma^*(s, a) = s \cup effects^+(a)$
- la distanza tra uno stato s e una proposizione p è data dalle seguenti relazioni:
 - $\Delta_0(s, p) = 0$ se $p \in s$
 - $\Delta_0(s, p) = \min_a \{1 + \Delta_0(s, precondition(a)) \mid p \in effects^+(a)\}$
 - $\Delta_0(s, p) = \infty$ se $\forall a \in A, p \notin effects^+(a)$

Ignorare effetti negativi: Ricerca in Avanti

- $\gamma^*(s, a) = s \cup effects^+(a)$
- la distanza tra uno stato s e una proposizione p è data dalle seguenti relazioni:
 - $\Delta_0(s, p) = 0$ se $p \in s$
 - $\Delta_0(s, p) = \min_a \{1 + \Delta_0(s, precondition(a)) \mid p \in effects^+(a)\}$
 - $\Delta_0(s, p) = \infty$ se $\forall a \in A, p \notin effects^+(a)$
- La distanza tra uno stato s e una congiunzione di proposizioni $g = p_1 \wedge \dots \wedge p_n$
 - $\Delta_0(s, g) = 0$ se $g \subseteq s$
 - $\Delta_0(s, g) = \sum_{p_i \in g} \Delta_0(s, p_i)$

Ignorare effetti negativi: Ricerca in Avanti

- $\gamma^*(s, a) = s \cup effects^+(a)$
- la distanza tra uno stato s e una proposizione p è data dalle seguenti relazioni:
 - $\Delta_0(s, p) = 0$ se $p \in s$
 - $\Delta_0(s, p) = \min_a \{1 + \Delta_0(s, precondition(a)) | p \in effects^+(a)\}$
 - $\Delta_0(s, p) = \infty$ se $\forall a \in A, p \notin effects^+(a)$
- La distanza tra uno stato s e una congiunzione di proposizioni $g = p_1 \wedge \dots \wedge p_n$
 - $\Delta_0(s, g) = 0$ se $g \subseteq s$
 - $\Delta_0(s, g) = \sum_{p_i \in g} \Delta_0(s, p_i)$
- la funzione euristica può essere quindi definita come:

$$h_0(s) = \Delta_0(s, g)$$

- h_0 non è ammissibile perché?
- è costosa perché richiede di risolvere un problema, se pur rilassato, per ogni stato visitato

Ignorare effetti negativi: Ricerca all'Indietro

- Regressione per mezzo della γ^{-1}
- Dato il sottogoal g attualmente in esame, esistono diverse azioni *rilevanti*
- Scegliere l'azione che porta al sottogoal più vicino allo stato iniziale

$$a_{best} \leftarrow \operatorname{argmin}\{\Delta_0(s_0, \gamma^{-1}(g, a)) \mid a \text{ è } \text{rilevante} \text{ (resolver) per } g\}$$

- l'euristica è sempre basata sulla distanza Δ_0 , quindi non è ammissibile
- il vero vantaggio è che è possibile precompilare le distanze tra s_0 e ogni possibile proposizione, il calcolo a runtime è quindi semplificato
 - dato un particolare goal $\gamma^{-1}(g, a)$ il suo costo è la somma dei costi delle proposizioni che lo compongono

- Consentono di ottenere soluzioni ottime
- **Safe pruning**: Se so che una soluzione ha al più costo k , posso scartare un nodo qualsiasi u tale che $h(u) > k$ senza correre il rischio di perdere soluzioni

$$h_1(s, g) = \Delta_1(s, g)$$

- data una proposizione p del goal e lo stato corrente s , la distanza di s da p è

$$\Delta_1(s, p) = \min_a \{1 + \max\{\Delta_1(s, q) \mid q \in \text{precond}(a), p \in \text{effects}^+(a)\}\}$$

- dato il goal g insieme di proposizioni

$$\Delta_1(s, g) = \max\{\Delta_1(s, p) \mid p \in g\}$$

- $\Delta_1(s, p)$ considera il costo di applicare un'azione a per ottenere p , tale costo è dominato dal costo peggiore per soddisfare le precondizioni di a , il costo per p con a è quindi il costo di $a + 1$. Tra tutte le azioni che producono p scelgo la più economica

- h_1 è poco informativa perché considera la distanza del goal come la massima distanza rispetto ad una singola proposizione p del goal
- si può raffinare la stima considerando la massima distanza di una coppia di proposizioni p, q del goal:

$$h_2(s, g) = \Delta_2(s, g)$$

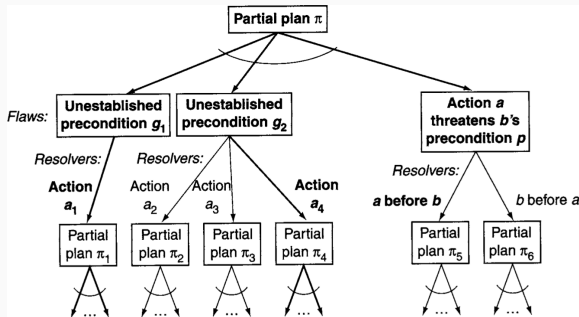
- $\Delta_2(s, p) = \Delta_1(s, p)$
 - $\Delta_2(s, \{p, q\}) = \min\{\begin{aligned} &\min_a \{1 + \Delta_2(s, \text{precond}(a)) \mid \{p, q\} \subseteq \text{effects}^+(a)\}, \\ &\min_a \{1 + \Delta_2(s, \{p\} \cup \text{precond}(a)) \mid q \in \text{effects}^+(a)\}, \\ &\min_a \{1 + \Delta_2(s, \{q\} \cup \text{precond}(a)) \mid p \in \text{effects}^+(a)\} \end{aligned}\}$
 - $\Delta_2(s, g) = \max_{\{p, q\}} \{\Delta_2(s, \{p, q\}) \mid \{p, q\} \subseteq g\}$
- h_2 è ammissibile ed è più informativa di h_1 , ma ovviamente è più costosa
 - l'idea può essere generalizzata ad un qualsiasi $k \leq |g|$

h_G : Graphplan come euristica

- Costruire un grafo di pianificazione senza considerare effetti negativi e mutua esclusione
- il costo di una proposizione p è la profondità del livello in cui p compare per la prima volta
- il costo di un goal è la profondità del livello in cui tutti gli atomi del goal compaiono tutti insieme per la prima volta
- h_G è simile ad h_2 , è ancora ammissibile, ma
 - è più facile calcolarla, e precompilarla
 - h_2 considera le azioni individualmente, ciascuna pesa uno, h_G considera livelli: due azioni allo stesso livello pesano 1
 - dipende solo dal dominio e dallo stato iniziale (potrei riusare lo stesso grafo di pianificazione per problemi simili (che condividono lo stesso stato iniziale) ma con goal differenti)
- Il pianificatore FF usa un'euristica simile

EURISTICHE PER ALGORITMI DI RICERCA NELLO SPAZIO DEI PIANI

Least-Commitment Planning come Ricerca AND/OR



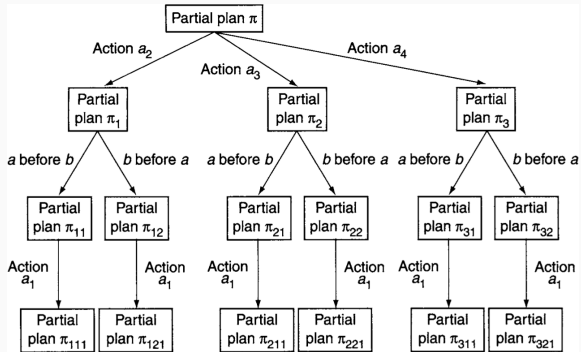
Due scelte non deterministiche:

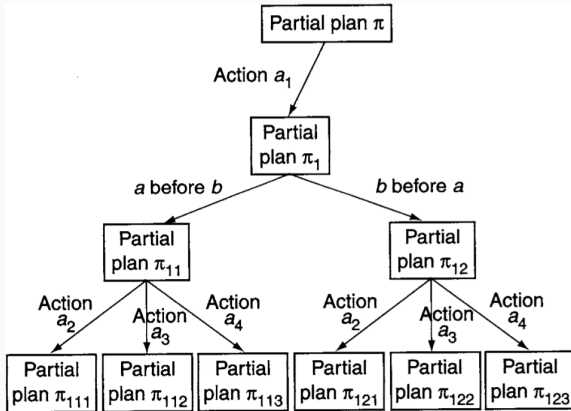
- prossimo **flaw** da considerare
- possibile **resolver** per il flaw selezionato

- I flaw rappresentano rami in AND perché devono essere risolti comunque tutti dal pianificatore
- I resolver rappresentano rami OR perché è sufficiente un solo resolver per flaw

Euristica per la selezione del flaw

- l'ordine con cui si selezionano i flaw è irrilevante per la “forma” della soluzione finale (che dipende dai resolver),
MA è fondamentale nel determinare il numero dei nodi:
 - La scelta di un flaw al posto di un altro può produrre un albero con molti più nodi, e quindi ad un calcolo meno efficiente della soluzione
- FAF (Fewer Alternatives First):
 - preferisce il flaw con il minor numero di resolvers
 - scopo: lavorando prima sui flaw con meno resolvers si esplorano nicchie dello spazio di ricerca e si limitano i costi di un eventuale backtracking
 - è più facile scoprire che un flaw non è risolubile





Euristica per la selezione di un resolver

- Scelto un flaw, esistono in genere diversi possibili resolvers per quel flaw
- Ciascun resolver porta ad una soluzione parziale (piano parziale)
- sia $\Omega = \{\pi_1, \dots, \pi_n\}$ l'insieme di tali piani parziali
- un'euristica per il resolver è calcolata sul piano parziale che questo produce, una possibile soluzione è:

$$\pi_{best} \leftarrow \operatorname{argmin}\{\eta_k(\pi) \mid \pi \in \Omega\}$$

Dove $\eta_k(\pi)$ stima per regressione, con al più k passi, la distanza del goal g da risolvere in π dallo stato iniziale s_0

Hierarchical Task Networks (HTN)

- Piani, anche se complessi, hanno sempre una certa **struttura regolare** (es. nella logistica una load sarà prima o poi seguita da una unload)
- Questa struttura può essere catturata in forma di piani astratti, o macro-azioni, o gerarchie di azioni
- I sottopiani corrispondenti a queste gerarchie sono di solito indipendenti
- HTN cerca di sfruttare quest'indipendenza cercando in uno spazio astratto, che non corrisponde allo spazio degli stati

Esempio: piano per andare ad una conferenza

- vai in aeroporto
- prendi l'aereo per la città della conferenza
- vai all'hotel della conferenza

Esempio: piano per andare ad una conferenza

- vai in aeroporto
 - Se hai tempo vai in aeroporto con i mezzi pubblici
 - Altrimenti, vai in macchina
- prendi l'aereo per la città della conferenza
- vai all'hotel della conferenza

Esempio: piano per andare ad una conferenza

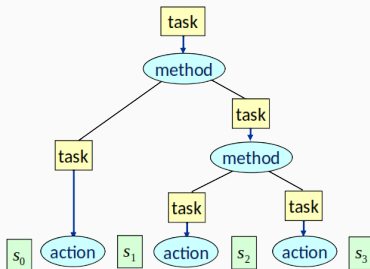
- vai in aeroporto
 - Se hai tempo vai in aeroporto con i mezzi pubblici
 - Altrimenti, vai in macchina
- prendi l'aereo per la città della conferenza
 - Fai il check-in
 - Passa la sicurezza
 - Vai al gate
- vai all'hotel della conferenza

Esempio: piano per andare ad una conferenza

- vai in aeroporto
 - Se hai tempo vai in aeroporto con i mezzi pubblici
 - Altrimenti, vai in macchina
- prendi l'aereo per la città della conferenza
 - Fai il check-in
 - Passa la sicurezza
 - Vai al gate
- vai all'hotel della conferenza
 - Se hai abbastanza soldi, chiama un taxi per andare all'hotel della conferenza
 - Altrimenti
 - compra il biglietto per i mezzi pubblici
 - recati alla fermata del mezzo pubblico

HTN: Elementi

- **Primitive** operator:
 - sono plan operator simili a quelli introdotti da STRIPS
 - sono direttamente eseguibili: modificano il mondo
- **Compound** operator (o **metodi**):
 - hanno precondizioni ed effetti
 - codificano la decomposizione di un task complesso in più sotto-task
- **soluzione: piano completamente istanziato**
(contiene solo operatori)



Algoritmo HTN (intuizione)

- **Input:** task astratto (non un goal!)
- **Repeat until** piano completamente istanziato
 - seleziona un metodo le cui condizioni di applicabilità sono soddisfatte
 - usa il metodo per espandere il piano corrente selezionando uno dei possibili sottopiani codificati nel metodo
- Per certi versi è simile al Least-Commitment Planning:
 - ad ogni iterazione rimuovo un flaw
 - un flaw è un subtask non ancora risolto

HTN: vantaggi e svantaggi

- I metodi codificano domain knowledge
- I metodi codificano problem solving knowledge
- E' teoreticamente più espressivo del planning classico
- E' Turing complete:
 - Per codificare loop è possibile invocare ricorsivamente lo stesso metodo
 - Per codificare scelte (if) è sufficiente definire decomposizioni alternative per lo stesso metodo

HTN: vantaggi e svantaggi

- I metodi codificano **domain knowledge**
- I metodi codificano **problem solving knowledge**
- E' teoreticamente più espressivo del planning classico
- E' Turing complete:
 - Per codificare loop è possibile invocare ricorsivamente lo stesso metodo
 - Per codificare scelte (if) è sufficiente definire decomposizioni alternative per lo stesso metodo

Contro:

- HTN è ancora **NP-Complete** nel caso peggiore
- Richiede un maggiore impegno di modellazione del problema perché non basta stabilire gli operatori, ma è necessario definire come un metodo può essere decomposto